



Escuela de ingeniería en computación
Ingeniería en computación
IC6821 - Diseño de Software

Avance 2 del proyecto

Diseño de arquitectura

Dayron Josué Porras Pérez
dayporras@estudiantec.cr
2023124693

Ian Sebastian Ugalde Muñoz
i.ugalde.1@estudiantec.cr
2024124076

Marcelo Gómez Cordero
mar_gomez@estudiantec.cr
2022136492

Javier Lee Liang
javi0409@estudiantec.cr
2020184824

José Emilio Alvarado Méndez
jos.alvarado@estudiantec.cr
2022163260

Provincia, Costa Rica
Febrero 2021

Índice general

1. Introducción	2
1.1. Explicación del problema	2
1.2. Explicación de la solución	3
1.3. Justificación de la solución	4
2. Desarrollo	5
2.1. Diseño de Persistencia	5
2.1.1. Modelo Conceptual	5
2.1.2. Modelo Lógico	6
2.2. Diseño de Arquitectura	13
2.2.1. Diagrama de Componentes	13
2.2.2. Diagrama de Clases	31
3. Conclusiones	34
Referencias	35

Capítulo 1

Introducción

El avance acelerado de las herramientas de inteligencia artificial generativa ha generado un desafío sin precedentes para la educación en programación. Plataformas como ChatGPT, Claude, Gemini y asistentes integrados como GitHub Copilot permiten que un estudiante obtenga fragmentos completos de código con una solicitud en lenguaje natural, lo que amenaza el propósito pedagógico de los ejercicios de programación.

Asimismo, algunos estudiantes, al sentirse abrumados por la falta de conocimiento o comprensión de los temas, pueden optar por no enfrentar el proceso de estudio de manera autónoma y recurrir a soluciones externas. En este sentido, la literatura indica que la reducción en el nivel de desempeño académico puede intensificar este problema, ya que los estudiantes con menor confianza en sus competencias presentan una mayor tendencia a replicar actividades o a utilizar servicios de terceros. (Laturette, Syahrani, y Octavica, 2025)

Si bien los tiempos cambian la mala conducta académica continúa siendo un problema persistente dentro del sector educativo, y el uso de herramientas como ChatGPT representa una manifestación reciente de esta situación (Gruenhagen y cols., 2024).

1.1. Explicación del problema

El problema central radica en que los entornos de desarrollo actuales — Visual Studio Code, PyCharm, IDLE, entre otros— no ofrecen ningún mecanismo nativo que limite el uso de IA o que prevenga el copiado masivo de código externo. Un estudiante puede abrir su editor favorito, copiar el código generado por una IA y pegarlo en el archivo de entrega sin ninguna fricción. Las herramientas de detección de plagio como Turnitin, aunque afirman contar con tecnología “altamente confiable” para distinguir texto humano de texto generado por IA (Turnitin, 2026), actúan de manera reactiva, analizando el producto final en lugar de intervenir durante el proceso de escritura.

En consecuencia, los profesores carecen de mecanismos proactivos que les

permitan asegurarse de que el código entregado fue escrito genuinamente por el estudiante dentro de un entorno controlado. Esta ausencia de trazabilidad y control durante el proceso de desarrollo pone en riesgo el aprendizaje y las competencias que los cursos de programación pretenden desarrollar. En este contexto surge lo que diversos autores advierten sobre la adopción de la inteligencia artificial en la educación, que no debe realizarse de forma acrítica, sino que requiere supervisión independiente, mecanismos regulatorios de seguridad y una participación significativa de docentes, estudiantes y comunidades (United Nations Educational, Scientific and Cultural Organization, 2023).

1.2. Explicación de la solución

La solución propuesta es un IDE (Entorno de Desarrollo Integrado) especializado para el lenguaje Python, desarrollado en C# con Windows Forms, que incorpora un conjunto de restricciones y mecanismos de control orientados a dificultar el uso de herramientas externas de IA durante la escritura de código. El sistema se denomina **IDE Académico de Python**.

El IDE implementa las siguientes capacidades principales:

- **Editor de código con restricción de copy-paste:** el estudiante puede escribir código Python libremente, pero las operaciones de copiar y pegar están bloqueadas dentro del editor, eliminando la principal vía de importación de código externo.
- **Firma encriptada por archivo:** cada archivo gestionado por el IDE recibe una firma digital única que permite detectar modificaciones realizadas fuera del sistema. Si el hash del archivo no coincide con la firma almacenada, el sistema alerta sobre una posible alteración externa.
- **Terminal interactiva integrada:** el estudiante puede ejecutar su código Python directamente dentro del IDE sin necesidad de herramientas externas.
- **Sistema de entrega y bitácora de versiones:** el estudiante entrega sus archivos directamente desde el IDE; el sistema genera una copia con marca temporal y registra cada versión en una bitácora histórica, dando trazabilidad completa al proceso.
- **Gestión de tareas y grupos:** los profesores crean tareas y asignan grupos de trabajo; los estudiantes ven únicamente las tareas que les corresponden según su grupo.

La arquitectura del sistema se articula en tres capas: una capa de presentación (Windows Forms), una capa de lógica de negocio con patrones de diseño bien definidos, y una capa de persistencia soportada por una base de datos relacional normalizada en Cuarta Forma Normal (4FN).

1.3. Justificación de la solución

La decisión de construir un IDE dedicado, en lugar de extender uno existente, responde a la necesidad de tener control absoluto sobre el entorno de edición. Los editores de texto convencionales exponen eventos de teclado y portapapeles a nivel del sistema operativo, lo que hace imposible bloquear el copy-paste de forma confiable sin modificar el núcleo de la aplicación.

El uso de una firma encriptada (hash SHA-256) resuelve el problema de la modificación externa del IDE: si un estudiante abre el archivo `.py` en otro editor y pega código, el hash que se almacena en la base de datos ya no coincidirá con el del archivo en disco, y el sistema lo detectará al intentar abrirlo nuevamente. Esto crea una capa de disuasión técnica sólida.

La bitácora de versiones aporta la trazabilidad necesaria al proceso de evaluación: el profesor puede ver no solo la entrega final, sino el historial completo de las versiones que los estudiantes han subido, lo que le permite analizar la progresión del trabajo del estudiante. Esta funcionalidad, combinada con la restricción de copy-paste y la firma encriptada, configura un sistema de capas de protección complementarias que, si bien no garantizan el 100% de la integridad académica, elevan significativamente el esfuerzo requerido para eludir los controles de plagio o copia de código.

El lenguaje C# y el entorno .NET se eligen por su madurez en el desarrollo de aplicaciones de escritorio en Windows, su excelente soporte para procesos del sistema operativo (necesario para lanzar el intérprete de Python) y la facilidad de integración con bases de datos relacionales mediante Entity Framework o ADO.NET.

Capítulo 2

Desarrollo

2.1. Diseño de Persistencia

2.1.1. Modelo Conceptual

Diagrama Entidad-Relación

El Diagrama Entidad-Relación (ER) de la Figura 2.1 representa las entidades del dominio del problema y sus relaciones con cardinalidad y participación. Se identifican siete entidades principales: **User**, **Profesor**, **Estudiante**, **Tarea**, **GrupoTrabajo**, **Entrega** y **ArchivoP**.

Las relaciones clave son:

- Un **User** es exactamente un **Profesor** o un **Estudiante** (especialización total y disjunta, cardinalidad 1:1).
- Un **Profesor** crea cero o muchas **Tareas** (1:N).
- Un **Profesor** crea cero o muchos **GrupoTrabajos** (1:N).
- Un **GrupoTrabajo** tiene exactamente una **Tarea** (N:1).
- Un **GrupoTrabajo** está en uno o más **Estudiantes** y un **Estudiante** puede pertenecer a muchos **GrupoTrabajos** (M:N).
- Un **Estudiante** tiene cero o muchas **Entregas** (1:N).
- Una **Entrega** tiene exactamente un **ArchivoP** (1:1).

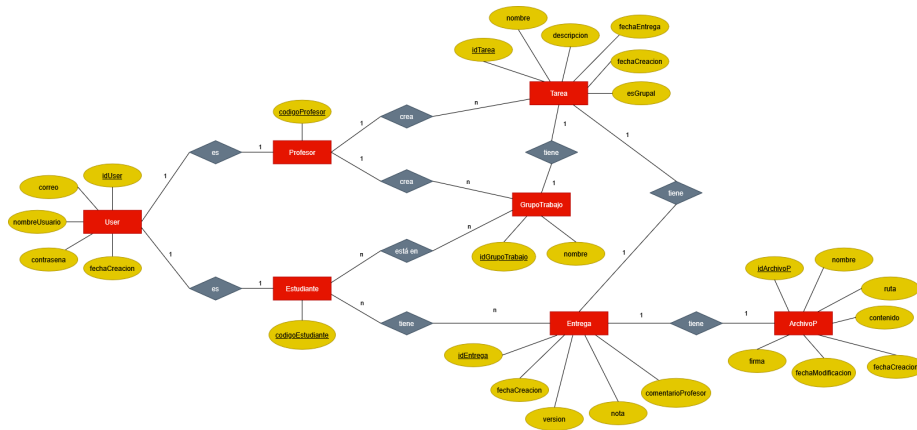


Figura 2.1: Diagrama Entidad-Relación del sistema IDE Académico de Python.

2.1.2. Modelo Lógico

Normalización a 4FN

Proceso de normalización. El modelo relacional derivado del diagrama conceptual fue sometido a un proceso de normalización progresiva hasta alcanzar la Cuarta Forma Normal (4FN), con el objetivo de eliminar redundancias y garantizar la integridad estructural de la persistencia.

- **1FN:** todos los atributos son atómicos; se elimina cualquier grupo repetido. Por ejemplo, el atributo `esGrupal` de `Tarea` es un valor booleano simple. Las relaciones M:N entre entidades se materializan en tablas de intersección explícitas (`EstudianteXGrupoTrabajo` y `EstudianteXEntrega`), eliminando cualquier posibilidad de listas embebidas.
- **2FN:** se eliminan dependencias parciales. Todas las tablas utilizan claves primarias simples, por lo que no existe posibilidad de dependencia parcial. En las tablas de intersección, los únicos atributos presentes son las claves foráneas que conforman la clave candidata compuesta.
- **3FN:** se eliminan dependencias transitivas. Los datos de `User` (correo, nombreUsuario, contraseña, fechaCreacion) no dependen de `codigoProfesor` ni de `codigoEstudiante`, sino de `idUser`; por eso `User` es una entidad separada. El mismo análisis se aplicó al resto de entidades sin encontrar cadenas transitivas.
- **BCNF:** todo determinante de una dependencia funcional no trivial es una superclave. En todas las tablas, el único determinante identificado es la clave primaria declarada; no existen determinantes alternativos que no sean superclaves.

- **4FN:** se eliminan dependencias multivaloradas no triviales. Tras revisar cada tabla, no se detectó ningún caso de este tipo. La tabla **EstudianteXGrupoTrabajo** aísla la asociación M:N entre estudiantes y grupos, y la tabla **EstudianteXEntrega** aísla la asociación M:N entre estudiantes y entregas grupales, garantizando que no existan dependencias multivaloradas en ninguna tabla. El esquema completo cumple la 4FN.

El modelo resultante en 4FN consta de las siguientes relaciones (las claves primarias están subrayadas y las foráneas marcadas con †):

1. **User**(idUser, correo, nombreUsuario, contrasena, fechaCreacion)
2. **Profesor**(codigoProfesor, idUser†)
3. **Estudiante**(codigoEstudiante, idUser†)
4. **Tarea**(idTarea, codigoProfesor†, nombre, descripcion, fechaCreacion, fechaEntrega, esGrupal)
5. **GrupoTrabajo**(idGrupoTrabajo, codigoProfesor†, idTarea†, nombre)
6. **EstudianteXGrupoTrabajo**(codigoEstudiante†, idGrupoTrabajo†)
7. **ArchivoP**(idArchivoP, nombre, ruta, contenido, fechaCreacion, fechaModificacion, firma)
8. **Entrega**(idEntrega, idTarea†, idArchivoP†, fechaCreacion, version, nota, comentarioProfesor)
9. **EstudianteXEntrega**(codigoEstudiante†, idEntrega†)

Diagrama Relacional

La Figura 2.2 presenta el esquema relacional normalizado del sistema. Cada tabla muestra su clave primaria (PK) y las claves foráneas (FK) que establecen los vínculos de integridad referencial entre entidades. Respecto al diagrama conceptual, la diferencia estructural más relevante es la materialización de las relaciones M:N en tablas de intersección (**EstudianteXGrupoTrabajo** y **EstudianteXEntrega**), y la incorporación explícita de **idTarea** como clave foránea en **GrupoTrabajo**, lo cual refleja que cada grupo de trabajo pertenece a una tarea específica y garantiza la integridad referencial en ambas direcciones de la relación.

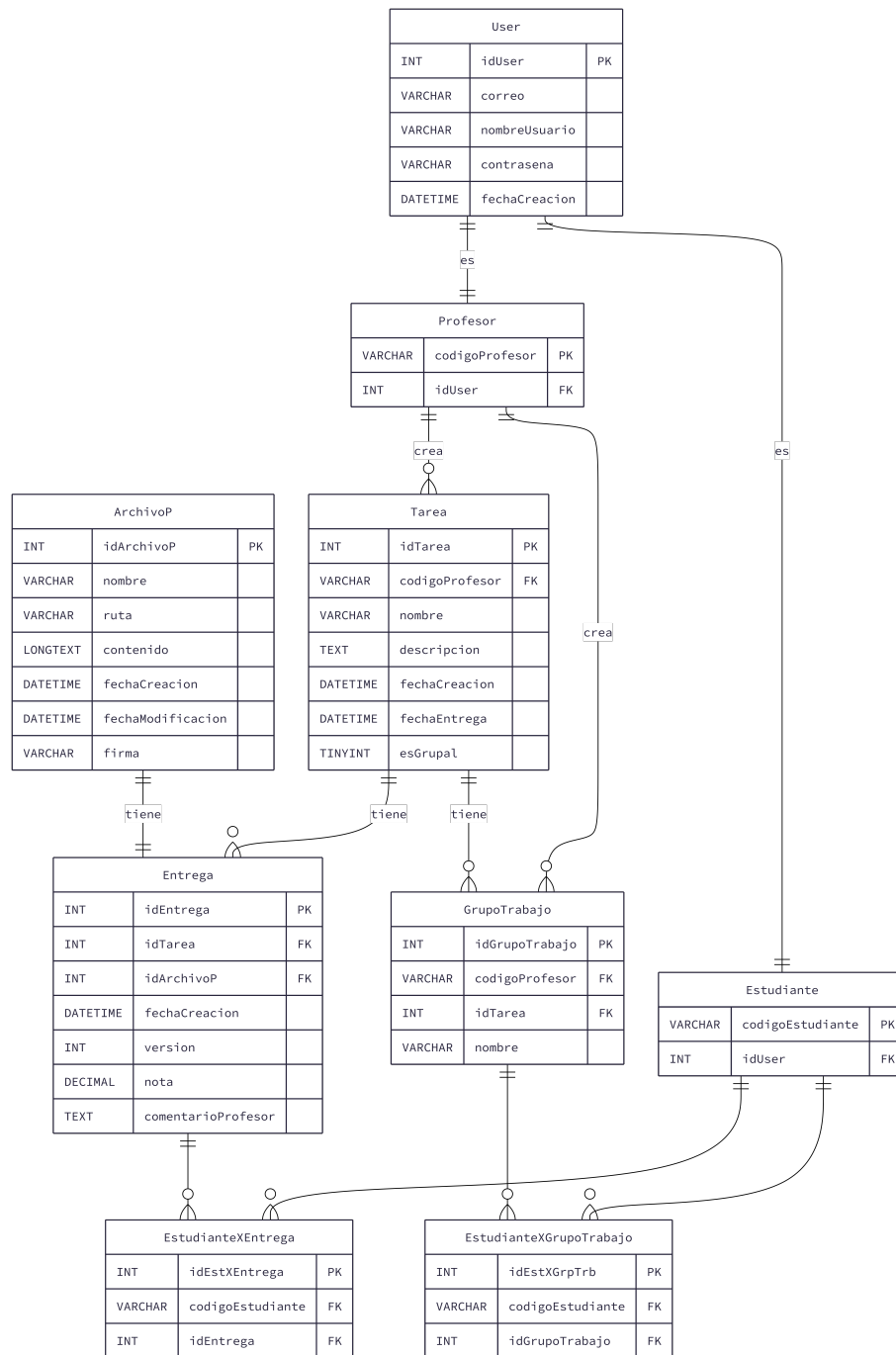


Figura 2.2: Diagrama Relacional normalizado en 4FN del sistema IDE Académico de Python.

Diccionario de Datos

A continuación se documenta cada entidad del modelo con una descripción de su rol en el sistema y una tabla de atributos que especifica el tipo de dato SQL, los valores válidos, la posibilidad de nulo y las restricciones aplicadas.

User. Representa a cualquier persona registrada en el sistema, independientemente de su rol. Almacena las credenciales de autenticación y los datos de identificación comunes a profesores y estudiantes. Cada usuario se especializa en exactamente uno de los dos roles disponibles.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
idUser	INT	Entero positivo autoincremental	No	PK, AUTO_INCREMENT
correo	VARCHAR(255)	Dirección de correo válida	No	NOT NULL, UNIQUE
nombreUsuario	VARCHAR(100)	Cadena no vacía	No	NOT NULL, UNIQUE
contrasena	VARCHAR(255)	Hash SHA-256 de la contraseña	No	NOT NULL
fechaCreacion	DATETIME	Fecha y hora válidas	No	NOT NULL, DEFAULT NOW()

Cuadro 2.1: Atributos de la entidad **User**.

Profesor. Especialización de **User** que representa a los docentes del sistema. Permite distinguir a los usuarios con permisos de creación y administración de tareas y grupos de trabajo. Su clave primaria es el código institucional del docente, independiente del identificador de usuario.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
codigoProfesor	VARCHAR(20)	Código institucional del docente	No	PK
idUser	INT	Referencia a User existente	No	FK $\rightarrow$ User(idUser), NOT NULL, UNIQUE

Cuadro 2.2: Atributos de la entidad **Profesor**.

Estudiante. Especialización de **User** que representa a los alumnos del sistema. Agrupa a los usuarios con permisos de acceso al editor, envío de entregas y participación en grupos de trabajo. Al igual que **Profesor**, mantiene una clave primaria propia para separar la identidad del rol.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
codigoEstudiante	VARCHAR(20)	Código institucional del alumno	No	PK
idUser	INT	Referencia a User existente	No	FK $\rightarrow$ User(idUser), NOT NULL, UNIQUE

Cuadro 2.3: Atributos de la entidad **Estudiante**.

Tarea. Representa una actividad académica publicada por un profesor. Contiene la descripción del enunciado, las fechas relevantes y la indicación de si el trabajo debe realizarse de forma grupal o individual. Una tarea puede tener asociados uno o más grupos de trabajo cuando la modalidad es grupal.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
idTarea	INT	Entero positivo autoincremental	No	PK, AUTO_INCREMENT
codigoProfesor	VARCHAR(20)	Referencia a Profesor existente	No	FK $\rightarrow$ Profesor(codigoProfesor), NOT NULL
nombre	VARCHAR(255)	Cadena no vacía	No	NOT NULL
descripcion	TEXT	Texto libre	Sí	—
fechaCreacion	DATETIME	Fecha y hora válidas	No	NOT NULL, DEFAULT NOW()
fechaEntrega	DATETIME	Fecha posterior a fecha-Creacion	Sí	—
esGrupal	TINYINT(1)	0 (individual) ó 1 (grupal)	No	NOT NULL

Cuadro 2.4: Atributos de la entidad **Tarea**.

GrupoTrabajo. Representa un equipo de estudiantes formado para una tarea grupal. Es creado por el profesor y agrupa a los alumnos que deben colaborar en una misma entrega. Cada grupo pertenece a exactamente una tarea, relación que se hace explícita mediante la clave foránea **idTarea**. La asociación entre grupos y estudiantes se gestiona a través de la tabla de intersección **EstudianteXGrupoTrabajo**.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
idGrupoTrabajo	INT	Entero positivo autoincremental	No	PK, AUTO_INCREMENT
codigoProfesor	VARCHAR(20)	Referencia a Profesor existente	No	FK $\rightarrow$ Profesor(codigoProfesor), NOT NULL
idTarea	INT	Referencia a Tarea existente	No	FK $\rightarrow$ Tarea(idTarea), NOT NULL
nombre	VARCHAR(255)	Cadena no vacía	No	NOT NULL

Cuadro 2.5: Atributos de la entidad **GrupoTrabajo**.

ArchivoP. Representa un archivo de código Python gestionado por el IDE. Almacena el contenido del código fuente, los metadatos del archivo (nombre, ruta, fechas) y la firma digital SHA-256 que permite detectar modificaciones realizadas fuera del entorno controlado. La presencia de la firma es opcional hasta el momento de la entrega.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
idArchivoP	INT	Entero positivo autoincremental	No	PK, AUTO_INCREMENT
nombre	VARCHAR(255)	Nombre de archivo válido (.py)	No	NOT NULL
ruta	VARCHAR(500)	Ruta en el sistema de archivos	No	NOT NULL
contenido	LONGTEXT	Código fuente Python	Sí	—
fechaCreacion	DATETIME	Fecha y hora válidas	No	NOT NULL, DEFAULT NOW()
fechaModificacion	DATETIME	Fecha posterior a fechaCreacion	Sí	—
firma	VARCHAR(64)	Hash SHA-256 en hexadecimal (64 chars)	Sí	—

Cuadro 2.6: Atributos de la entidad **ArchivoP**.

Entrega. Registra un evento de entrega de tarea realizado por uno o más estudiantes. Asocia un archivo de código con una tarea específica e incluye la versión de la entrega, la nota asignada y el comentario del profesor. Cada nueva entrega de la misma tarea genera un registro independiente con número de versión incremental, construyendo así la bitácora de historial requerida por el sistema.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
idEntrega	INT	Entero positivo autoincremental	No	PK, AUTO_INCREMENT
idTarea	INT	Referencia a Tarea existente	No	FK → Tarea(idTarea), NOT NULL
idArchivoP	INT	Referencia a ArchivoP existente	No	FK → ArchivoP(idArchivoP), NOT NULL
fechaCreacion	DATETIME	Fecha y hora válidas	No	NOT NULL, DEFAULT NOW()
version	INT	Entero positivo ≥ 1	No	NOT NULL
nota	DECIMAL(5,2)	0.00 a 100.00	Sí	CHECK (nota BETWEEN 0 AND 100)
comentarioProfesor	TEXT	Texto libre	Sí	—

Cuadro 2.7: Atributos de la entidad **Entrega**.

EstudianteXGrupoTrabajo. Tabla de intersección que resuelve la relación M:N entre **Estudiante** y **GrupoTrabajo**. Un estudiante puede pertenecer a va-

rios grupos y un grupo puede estar integrado por varios estudiantes. La combinación de sus dos claves foráneas es única para evitar registros duplicados.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
idEstXGrpTrb	INT	Entero positivo autoincremental	No	PK, AUTO.INCREMENT
codigoEstudiante	VARCHAR(20)	Referencia a Estudiante existente	No	FK → Estudiante(codigoEstudiante), NOT NULL
idGrupoTrabajo	INT	Referencia a GrupoTrabajo existente	No	FK → GrupoTrabajo(idGrupoTrabajo), NOT NULL
UNIQUE(codigoEstudiante, idGrupoTrabajo)				

Cuadro 2.8: Atributos de la entidad **EstudianteXGrupoTrabajo**.

EstudianteXEntrega. Tabla de intersección que resuelve la relación M:N entre **Estudiante** y **Entrega**. Permite que una misma entrega grupal esté asociada a varios estudiantes y que un estudiante figure en múltiples entregas a lo largo del curso. La combinación de sus dos claves foráneas es única para evitar registros duplicados.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
idEstXEntrega	INT	Entero positivo autoincremental	No	PK, AUTO.INCREMENT
codigoEstudiante	VARCHAR(20)	Referencia a Estudiante existente	No	FK → Estudiante(codigoEstudiante), NOT NULL
idEntrega	INT	Referencia a Entrega existente	No	FK → Entrega(idEntrega), NOT NULL
UNIQUE(codigoEstudiante, idEntrega)				

Cuadro 2.9: Atributos de la entidad **EstudianteXEntrega**.

2.2. Diseño de Arquitectura

2.2.1. Diagrama de Componentes

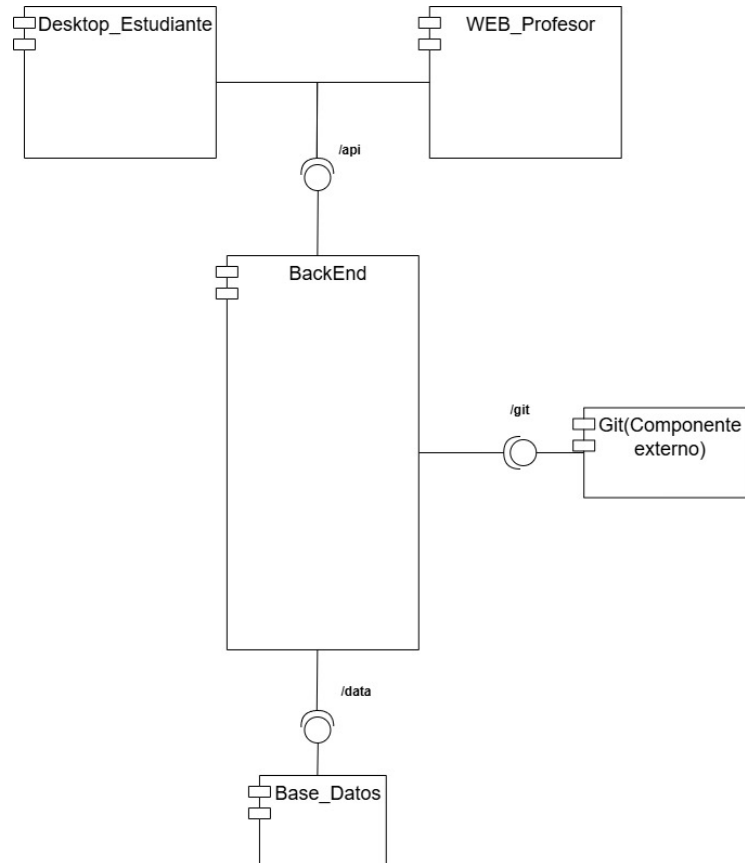


Figura 2.3: Diagrama de componentes del sistema IDE Académico de Python.

Descripción de Componentes

1. DeskTop_Estudiante: El componente Desktop_Estudiante representa la aplicación de escritorio utilizada por los estudiantes para interactuar con el sistema. Su responsabilidad es proporcionar las funcionalidades de acceso, edición de código, administración de archivos, ejecución de programas y envío de entregas. Este componente no publica interfaces para otros componentes de la arquitectura; su papel es únicamente consumir la interfaz /api expuesta por el componente BackEnd.

2. Web_Profesor: El componente WEB_Profesor corresponde a la aplicación web usada por los profesores para administrar las actividades académicas dentro del sistema. Su rol consiste en permitir la autenticación del profesor, la creación y administración de tareas, la asignación de grupos, la consulta de entregas y el proceso de revisión y calificación. Este componente no publica interfaces para otros componentes de la arquitectura; únicamente consume la interfaz /api publicada por el componente BackEnd.

3. BackEnd: El componente BackEnd es el núcleo del sistema y funciona como una fachada que concentra la lógica de negocio y el acceso a los servicios internos y externos. Su responsabilidad es recibir las solicitudes provenientes de Desktop_Estudiente y WEB_Profesor, validarlas, procesarlas y coordinar la comunicación con los componentes Base_Datos y Git. Este componente publica la interfaz "/api", que agrupa los servicios de autenticación, gestión de tareas, grupos, archivos, ejecución, entregas y bitácora. Internamente consume las interfaces /data y /git.

Interfaz Publicada: /api

Métodos a publicar por esta interfaz:

3.1. POST /api/auth/login Descripción: autentica a un usuario dentro del sistema.

Entrada:

```
{
  "correo": "ana@universidad.edu",
  "contrasena": "123456"
}
```

Salida:

```
{
  "idUsuario": 15,
  "nombre": "Ana López",
  "rol": "ESTUDIANTE",
  "token": "jwt-token-ejemplo"
}
```

Ejemplo detallado: Una estudiante abre la aplicación de escritorio y escribe su correo y contraseña. El cliente envía esta información a /api/auth/login. El backend valida las credenciales consultando los datos persistidos del usuario. Si la autenticación es exitosa, devuelve el identificador del usuario, su nombre, su rol y un token de sesión que permitirá realizar operaciones posteriores.

3.2. POST /api/auth/register Descripción: registra un nuevo usuario en el sistema.

Entrada:

```
{
  "correo": "nuevo@universidad.edu",
  "nombre": "Carlos Pérez",
  "nombreUsuario": "cperez",
  "contrasena": "abc123",
  "rol": "ESTUDIANTE"
}
```

Salida:

```
{
  "idUsuario": 21,
  "estado": "registrado"
}
```

Ejemplo detallado: Durante el proceso de incorporación de un nuevo estudiante, el cliente envía sus datos básicos al backend. El sistema valida el formato de correo, la consistencia de la contraseña y la inexistencia previa del usuario. Si todo es correcto, registra la cuenta y devuelve el identificador asignado.

3.3. GET /api/tareas Descripción: consulta las tareas asociadas a un usuario.

Entrada:

```
{
  "usuarioId": 15,
  "rol": "ESTUDIANTE"
}
```

Salida:

```
[
  {
    "idTarea": 101,
    "nombre": "Práctica 1",
    "descripcion": "Resolver ejercicios de recursividad",
    "fechaEntrega": "2026-05-10",
    "esGrupal": false
  }
]
```

Ejemplo detallado: Después de iniciar sesión, el estudiante entra al módulo de tareas. El cliente solicita al backend la lista de tareas disponibles para ese usuario. El backend consulta la base de datos y responde con todas las tareas vigentes relacionadas con el estudiante.

3.4. POST /api/tareas Descripción: crea una nueva tarea.

Entrada:

```
{
  "nombre": "Proyecto 1",
  "descripcion": "Implementar árbol binario",
  "fechaEntrega": "2026-05-20",
  "esGrupal": true,
  "profesorId": 3
}
```

Salida:

```
{
  "idTarea": 120,
  "estado": "creada"
}
```

Ejemplo detallado: El profesor ingresa al panel web y completa el formulario de creación de una nueva tarea. El sistema recibe el nombre, la descripción, la fecha de entrega y si la actividad será grupal. El backend valida que el usuario tenga rol de profesor, registra la tarea y devuelve su identificador.

3.5. PUT /api/tareas/{idTarea} Descripción: modifica la información de una tarea existente.

Entrada:

```
{
  "nombre": "Proyecto 1 - corregido",
  "descripcion": "Implementar árbol binario balanceado",
  "fechaEntrega": "2026-05-25",
  "esGrupal": true
}
```

Salida:

```
{
  "idTarea": 120,
  "estado": "actualizada"
}
```

Ejemplo detallado: El profesor detecta que debe ampliar el plazo de entrega de una tarea ya publicada. Desde la web modifica los datos y el cliente envía la nueva información al backend. El sistema actualiza el registro y devuelve una confirmación.

3.6. POST /api/tareas/{idTarea}/grupos Descripción: asigna o registra un grupo de trabajo para una tarea grupal.

Entrada:

```
{
  "nombreGrupo": "Equipo Alpha",
  "miembros": [15, 16, 17]
}
```

Salida:

```
{
  "idGrupo": 9,
  "estado": "grupo_registrado"
}
```

Ejemplo detallado: Para una actividad grupal, el profesor define los equipos desde la plataforma web. El backend crea el grupo asociado a la tarea y registra a los estudiantes que lo integran.

3.7. GET /api/tareas/{idTarea}/entregas Descripción: consulta las entregas asociadas a una tarea.

Entrada: no requiere cuerpo; el identificador de la tarea viaja en la ruta.

Salida:

```
[
  {
    "idEntrega": 501,
    "estudiante": "Ana López",
    "fechaHora": "2026-04-22T18:40:00",
    "version": 1
  }
]
```

Ejemplo detallado: El profesor abre la vista de seguimiento de una tarea. El sistema devuelve todas las entregas registradas para dicha actividad, permitiendo revisar qué estudiantes ya enviaron evidencia.

3.8. POST /api/archivos Descripción: crea o registra un archivo asociado a un estudiante.

Entrada:

```
{
  "estudianteId": 15,
  "nombre": "main.py",
  "ruta": "/proyecto1/main.py",
  "contenido": "print('Hola mundo')"
}
```

Salida:

```
{
  "idArchivo": 88,
  "estado": "archivo_creado"
}
```

Ejemplo detallado: El estudiante crea un nuevo archivo desde el editor de código. La aplicación de escritorio envía el nombre, ruta y contenido al backend, que registra el archivo y lo deja disponible para ediciones posteriores.

3.9. GET /api/archivos/{idArchivo} Descripción: abre o recupera el contenido de un archivo.

Entrada: no requiere cuerpo; el identificador viaja en la ruta.

Salida:

```
{
  "idArchivo": 88,
  "nombre": "main.py",
  "ruta": "/proyecto1/main.py",
  "contenido": "print('Hola mundo')"
}
```

Ejemplo detallado: Cuando el estudiante vuelve a abrir un archivo que había guardado anteriormente, el cliente solicita su contenido al backend y este responde con toda la información necesaria para cargarlo en el editor.

3.10. PUT /api/archivos/{idArchivo} Descripción: actualiza el contenido de un archivo existente.

Entrada:

```
{
  "contenido": "print('Hola mundo modificado')"
}
```

Salida:

```
{
  "idArchivo": 88,
  "estado": "archivo_actualizado"
}
```

Ejemplo detallado: Mientras el estudiante trabaja en el editor, guarda cambios sobre un archivo ya existente. El backend persiste la nueva versión y devuelve confirmación de actualización.

3.11. POST /api/archivos/{idArchivo}/firma Descripción: genera y registra la firma digital asociada a un archivo.

Entrada:

```
{
  "idArchivo": 88
}
```

Salida:

```
{
  "firma": "A94F5C128B...",
  "estado": "firma_generada"
}
```

Ejemplo detallado: Antes de una entrega, el sistema puede generar una firma o hash del archivo como mecanismo de integridad. El backend procesa el contenido, calcula la firma y la devuelve para su almacenamiento o validación posterior.

3.12. POST /api/ejecucion Descripción: ejecuta el código contenido en un archivo.

Entrada:

```
{
  "archivoId": 88,
  "entrada": ""
}
```

Salida:

```
{
  "salida": "Hola mundo",
  "errores": "",
  "duracion": 35,
  "codigoSalida": 0
}
```

Ejemplo detallado: El estudiante presiona el botón “Ejecutar” desde la aplicación de escritorio. El cliente envía el identificador del archivo que debe ejecutarse. El backend procesa la solicitud, ejecuta el programa y retorna la salida estándar, los errores si existen, la duración y el código de salida.

3.13. POST /api/entregas Descripción: registra una nueva entrega de tarea.

Entrada:

```
{
  "tareaId": 101,
  "estudianteId": 15,
  "archivoId": 88,
  "nota": 0,
  "comentarioProfesor": ""
}
```

Salida:

```
{
  "idEntrega": 501,
  "estado": "entrega_registrada",
  "version": 1
}
```

Ejemplo detallado: Una vez finalizado su trabajo, el estudiante selecciona el archivo correspondiente y confirma la entrega. El backend verifica que la tarea exista, esté vigente y que el estudiante esté autorizado para entregarla. Después registra la entrega y devuelve el identificador generado.

3.14. GET /api/entregas/{idEntrega}/firma Descripción: obtiene la firma o identificador de integridad de una entrega.

Entrada: no requiere cuerpo.

Salida:

```
{
  "idEntrega": 501,
  "firma": "D84K11FF991..."
}
```

Ejemplo detallado: Durante una revisión, el profesor o el sistema puede requerir verificar la integridad de una entrega. El backend obtiene la firma relacionada y la devuelve para comparación o auditoría.

3.15. GET /api/entregas/{idEntrega} Descripción: consulta el detalle de una entrega específica.

Entrada: no requiere cuerpo.

Salida:

```
{
  "idEntrega": 501,
  "fechaHora": "2026-04-22T18:40:00",
  "version": 1,
  "archivo": {
    "idArchivo": 88,
    "nombre": "main.py"
  },
}
```

```
"nota": 90,
"comentarioProfesor": "Buen trabajo"
}
```

Ejemplo detallado: El profesor selecciona una entrega concreta desde la lista de entregas. El backend devuelve toda la información relevante para que pueda revisarla y calificarla.

3.16. GET /api/bitacora Descripción: consulta el historial de entregas asociado a una tarea y a un estudiante.

Entrada:

```
{
  "tareaId": 101,
  "estudianteId": 15
}
```

Salida:

```
[
  {
    "idEntrega": 498,
    "fechaHora": "2026-04-20T10:00:00",
    "version": 1
  },
  {
    "idEntrega": 501,
    "fechaHora": "2026-04-22T18:40:00",
    "version": 2
  }
]
```

Ejemplo detallado: El estudiante o el profesor puede consultar el historial de versiones de una entrega para una tarea específica. El backend recupera todos los registros asociados y los devuelve ordenados cronológicamente.

3.17. GET /api/bitacora/ultima-version Descripción: obtiene la versión más reciente de una entrega para una tarea y un estudiante.

Entrada:

```
{
  "tareaId": 101,
  "estudianteId": 15
}
```

Salida:

```
{
  "idEntrega": 501,
  "version": 2,
  "fechaHora": "2026-04-22T18:40:00"
}
```

Ejemplo detallado: Cuando el profesor quiere revisar únicamente la última entrega disponible de un estudiante, el sistema consulta la bitácora y devuelve el registro más reciente.

4. Base.Datos: El componente Base_Datos se encarga del almacenamiento persistente de la información del sistema. Su rol es conservar los datos relacionados con usuarios, tareas, grupos, archivos, entregas y bitácoras, permitiendo que el backend recupere y actualice esta información de forma consistente. Este componente publica la interfaz /data, la cual abstrae las operaciones de persistencia y evita que el componente BackEnd dependa directamente de detalles del motor de base de datos utilizado.

Interfaz Publicada: /data

Métodos a publicar por esta interfaz:

4.1. GET /data/usuarios/{id} Descripción: recupera la información de un usuario por su identificador.

Entrada: identificador del usuario en la ruta.

Salida:

```
{
  "id": 15,
  "correo": "ana@universidad.edu",
  "nombre": "Ana López",
  "nombreUsuario": "alopez",
  "rol": "ESTUDIANTE"
}
```

Ejemplo detallado: Durante el proceso de autenticación o consulta de perfil, el backend necesita recuperar la información de un usuario específico. La base de datos responde con los datos persistidos asociados al identificador solicitado.

4.2. POST /data/usuarios Descripción: guarda un nuevo usuario.

Entrada:

```
{
  "correo": "nuevo@universidad.edu",
  "nombre": "Carlos Pérez",
  "nombreUsuario": "cperez",
  "contrasena": "abc123",
  "rol": "ESTUDIANTE"
}
```

Salida:

```
{
  "id": 21,
  "estado": "usuario_guardado"
}
```

Ejemplo detallado: En el registro de un nuevo usuario, el backend envía todos los datos validados a la base de datos, la cual persiste el nuevo registro y devuelve el identificador asignado.

4.3. GET /data/tareas Descripción: recupera tareas según filtros de búsqueda.

Entrada:

```
{
  "usuarioId": 15,
  "rol": "ESTUDIANTE"
}
```

Salida:

```
[
  {
    "idTarea": 101,
    "nombre": "Práctica 1",
    "fechaEntrega": "2026-05-10",
    "esGrupal": false
  }
]
```

Ejemplo detallado: El backend solicita a la base de datos todas las tareas relacionadas con un estudiante o con un profesor. La base devuelve el conjunto correspondiente.

4.4. POST /data/tareas Descripción: guarda una tarea nueva.

Entrada:

```
{
  "nombre": "Proyecto 1",
  "descripcion": "Implementar árbol binario",
  "fechaEntrega": "2026-05-20",
  "esGrupal": true,
  "profesorId": 3
}
```

Salida:


```
{
  "idTarea": 120,
  "estado": "tarea_guardada"
}
```

Ejemplo detallado: Después de validar una nueva tarea, el backend la envía a la capa de datos para que sea almacenada permanentemente.

4.5. POST /data/grupos Descripción: registra un grupo de trabajo.

Entrada:

```
{
  "nombreGrupo": "Equipo Alpha",
  "tareaId": 120,
  "miembros": [15, 16, 17]
}
```

Salida:

```
{
  "idGrupo": 9,
  "estado": "grupo_guardado"
}
```

Ejemplo detallado: Cuando el profesor asigna grupos a una tarea, el backend envía la composición del grupo a la base de datos para dejarla almacenada.

4.6. POST /data/archivos Descripción: guarda un archivo.

Entrada:

```
{
  "estudianteId": 15,
  "nombre": "main.py",
  "ruta": "/proyecto1/main.py",
  "contenido": "print('Hola mundo')"
}
```

Salida:

```
{
  "idArchivo": 88,
  "estado": "archivo_guardado"
}
```

Ejemplo detallado: Cada vez que el estudiante crea o registra un archivo, el backend envía la información correspondiente a la base de datos para dejarla persistida.

4.7. GET /data/archivos/{idArchivo} Descripción: recupera un archivo por su identificador.

Entrada: identificador del archivo en la ruta.

Salida:

```
{
  "idArchivo": 88,
  "nombre": "main.py",
  "ruta": "/proyecto1/main.py",
  "contenido": "print('Hola mundo')"
}
```

Ejemplo detallado: Cuando se necesita abrir un archivo guardado previamente, el backend solicita la información a la base de datos y esta devuelve su contenido.

4.8. PUT /data/archivos/{idArchivo} Descripción: actualiza el contenido o metadatos de un archivo.

Entrada:

```
{
  "contenido": "print('Hola mundo modificado')"
}
```

Salida:

```
{
  "idArchivo": 88,
  "estado": "archivo_actualizado"
}
```

Ejemplo detallado: Al guardar cambios en el editor, el backend manda la nueva versión del archivo a la base de datos para sobrescribir el contenido anterior.

4.9. POST /data/entregas Descripción: registra una nueva entrega.

Entrada:

```
{
  "tareaId": 101,
  "estudianteId": 15,
  "archivoId": 88,
  "version": 1,
  "nota": 0,
  "comentarioProfesor": ""
}
```

Salida:

```
{
  "idEntrega": 501,
  "estado": "entrega_guardada"
}
```

Ejemplo detallado: Una vez que el backend valida una entrega, solicita a la base de datos almacenarla como nueva versión asociada a una tarea y a un estudiante.

4.10. GET /data/entregas Descripción: recupera entregas según filtros como tarea o estudiante.

Entrada:

```
{
  "tareaId": 101
}
```

Salida:

```
[
  {
    "idEntrega": 501,
    "estudianteId": 15,
    "fechaHora": "2026-04-22T18:40:00",
    "version": 1
  }
]
```

Ejemplo detallado: Para listar las entregas de una tarea, el backend consulta la base de datos con el identificador de la actividad y obtiene todas las entregas relacionadas.

4.11. GET /data/entregas/{idEntrega} Descripción: recupera los datos de una entrega específica.

Entrada: identificador de la entrega en la ruta.

Salida:

```
{
  "idEntrega": 501,
  "tareaId": 101,
  "estudianteId": 15,
  "archivoId": 88,
  "nota": 90,
  "comentarioProfesor": "Buen trabajo"
}
```

Ejemplo detallado: Cuando el profesor selecciona una entrega concreta, el backend solicita a la base de datos el detalle completo del registro para mostrarlo en pantalla.

4.12. POST /data/bitacora Descripción: registra un evento en la bitácora de entregas.

Entrada:

```
{
  "idEntrega": 501,
  "fechaHora": "2026-04-22T18:40:00",
  "version": 1,
  "idArchivo": 88
}
```

Salida:

```
{
  "estado": "registro_bitacora_guardado"
}
```

Ejemplo detallado: Cada vez que una entrega es aceptada, el backend registra también un evento en la bitácora para conservar el historial de versiones.

4.13. GET /data/bitacora Descripción: obtiene el historial de entregas de un estudiante para una tarea.

Entrada:

```
{
  "tareaId": 101,
  "estudianteId": 15
}
```

Salida:

```
[
  {
    "idEntrega": 498,
    "version": 1,
    "fechaHora": "2026-04-20T10:00:00"
  },
  {
    "idEntrega": 501,
    "version": 2,
    "fechaHora": "2026-04-22T18:40:00"
  }
]
```

Ejemplo detallado: Cuando se desea mostrar el historial de envíos de un estudiante, el backend consulta la bitácora persistida y obtiene todas las versiones registradas.

4.14. GET /data/usuarios Descripción: busca un usuario a partir de un criterio de autenticación, como el correo electrónico, para permitir su validación durante el inicio de sesión.

Entrada:

```
{
  "correo": "ana@universidad.edu"
}
```

Salida:

```
{
  "id": 15,
  "correo": "ana@universidad.edu",
  "nombre": "Ana López",
  "nombreUsuario": "alopez",
  "contrasenaHash": "hash-ejemplo",
  "rol": "ESTUDIANTE"
}
```

Ejemplo detallado: Cuando un usuario intenta iniciar sesión desde POST /api/auth/login, el backend necesita buscar en la capa de datos el registro asociado al correo ingresado. La base de datos devuelve la información persistida del usuario, incluyendo sus credenciales protegidas y su rol, para que el backend pueda validar la autenticación. Esto completa la trazabilidad con el método de login en /api.

4.15. PUT /data/tareas/{idTarea} Descripción: actualiza la información de una tarea existente en la base de datos.

Entrada:

```
{
  "nombre": "Proyecto 1 - corregido",
  "descripcion": "Implementar árbol binario balanceado",
  "fechaEntrega": "2026-05-25",
  "esGrupal": true
}
```

Salida:

```
{
  "idTarea": 120,
  "estado": "tarea_actualizada"
}
```

Ejemplo detallado: Si el profesor modifica una tarea desde PUT /api/tareas/{idTarea}, el backend debe reflejar ese cambio en la persistencia. Para ello envía a /data/tareas/{idTarea} los nuevos valores de la tarea. La base de datos actualiza el registro correspondiente y responde con una confirmación de que la tarea fue modificada correctamente. Esto hace consistente la edición de tareas entre /api y /data.

4.16. PUT /data/archivos/{idArchivo}/firma Descripción: guarda o actualiza la firma digital asociada a un archivo, como parte del mecanismo de verificación de integridad.

Entrada:

```
{
  "firma": "A94F5C128B..."
}
```

Salida:

```
{
  "idArchivo": 88,
  "estado": "firma_actualizada"
}
```

Ejemplo detallado: Después de que el backend genera la firma del archivo mediante POST /api/archivos/{idArchivo}/firma, necesita persistir ese valor en la capa de datos. Entonces envía la firma calculada a /data/archivos/{idArchivo}/firma. La base de datos actualiza el archivo asociado con la nueva firma y devuelve el estado de la operación. Esto deja claro dónde se almacena la firma que luego servirá para validar la integridad del archivo.

4.17. GET /data/entregas/{idEntrega}/firma Descripción: recupera la firma de integridad asociada a una entrega específica.

Entrada: no requiere cuerpo; el identificador de la entrega viaja en la ruta.

Salida:

```
{
  "idEntrega": 501,
  "firma": "D84K11FF991..."
}
```

Ejemplo detallado: Cuando el profesor o el sistema solicita la firma de una entrega mediante GET /api/entregas/{idEntrega}/firma, el backend necesita consultar esa información en la capa de datos. La base de datos busca la firma asociada a la entrega solicitada y devuelve el valor almacenado, permitiendo que el backend lo use para verificación o auditoría. Así queda respaldado el método correspondiente de /api.

4.18. GET /data/bitacora/ultima-version Descripción: obtiene el registro más reciente de la bitácora de entregas para una tarea y un estudiante determinados.

Entrada:

```
{
  "tareaId": 101,
  "estudianteId": 15
}
```

Salida:

```
{
  "idEntrega": 501,
  "version": 2,
  "fechaHora": "2026-04-22T18:40:00"
}
```

Ejemplo detallado: Cuando el backend recibe la solicitud GET `/api/bitacora/ultima-version`, necesita consultar en la capa de datos cuál es la última versión registrada para un estudiante en una tarea específica. La base de datos filtra la bitácora correspondiente, identifica el registro más reciente y lo devuelve al backend con el número de versión y la fecha de envío. Esto hace consistente la consulta especializada de última versión entre ambas capas.

5. Git: El componente Git corresponde a un servicio externo de control de versiones integrado al sistema. Su función es mantener historial de cambios de archivos y apoyar la trazabilidad de las versiones manipuladas por el sistema, particularmente aquellas asociadas a entregas y archivos de estudiantes. Este componente publica la interfaz `/git`, la cual es consumida únicamente por el BackEnd.

Interfaz Publicada: `/git`

Métodos a publicar por esta interfaz:

5.1. POST `/git/repos` Descripción: crea un repositorio asociado a un usuario, proyecto o tarea.

Entrada:

```
{
  "nombreRepo": "proyecto1-estudiante-15",
  "propietario": "ana"
}
```

Salida:

```
{
  "repoId": "repo_15",
  "url": "https://git.example.com/repo_15"
}
```

Ejemplo detallado: Cuando el sistema necesita inicializar el espacio de trabajo versionado de un estudiante o de una tarea, el backend solicita la creación de un repositorio. Git devuelve el identificador y la URL del recurso creado.

5.2. POST `/git/commits` Descripción: registra un commit asociado a cambios realizados en el repositorio.

Entrada:

```
{
  "repoId": "repo_15",
  "mensaje": "Entrega final de práctica 1"
}
```

Salida:

```
{
  "commitId": "c1a45f",
  "estado": "commit_registrado"
}
```

Ejemplo detallado: Una vez sincronizado el archivo de una entrega, el backend puede solicitar que se registre un commit descriptivo. Git guarda el cambio en el historial y devuelve el identificador del commit generado.

2.2.2. Diagrama de Clases

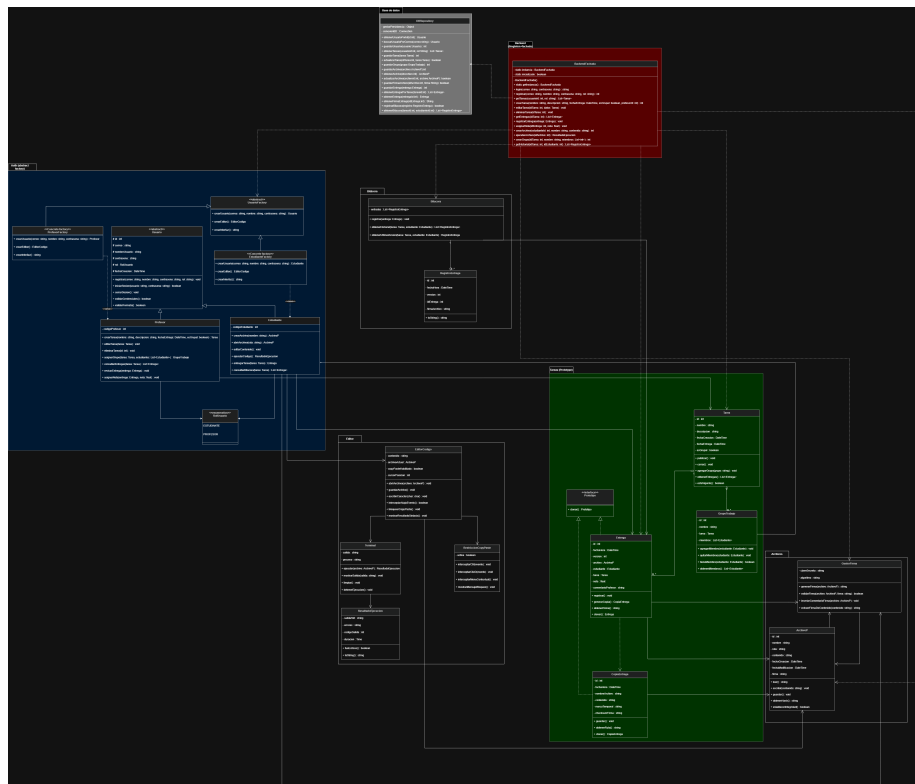


Figura 2.4: Diagrama de clases del sistema IDE Académico de Python.

Patrones de diseño: justificación e implementación

1. Facade: Se decidió implementar el patrón de “Fachada” en el sistema mediante la clase “BackEndFachada”, esto para evitar que los clientes (Desktop_Estudiante y Web_Profesor) interactúen directamente con las muchas clases como “Tarea”, “Entrega”, etc. Sin este patrón los clientes tendrían un alto acoplamiento con la lógica interna.

“BackEndFachada” cumple la función de ser el único punto de acceso al sistema. Esta clase expone métodos de alto nivel como “login”, “registro”, “crearTarea”, “registrarEntrega”, etc, encapsulando la coordinación entre las diferentes clases internas. También, la fachada se encarga de interactuar con “DBRepository” para la coordinación y persistencia de la base de datos.

2. Factory: El patrón Factory Method se utiliza para delegar la responsabilidad de instanciación a una clase especializada en lugar de hacerlo directamente en el código cliente, desacoplando la creación de objetos de su uso.

Para el sistema, se incorporó este patrón para manejar la creación de diferentes tipos de objetos relacionados con el usuario y componentes del entorno, evitando que otras clases dependan directamente de constructores específicos.

La implementación se observa en la clase “UsuarioFactory”, la cual define métodos como “crearUsuario”, “crearEditor” y “crearInterfaz”. Esta clase centraliza la lógica de creación, permitiendo que otras partes del sistema soliciten objetos sin conocer los detalles de su construcción.

3. Prototype: El patrón Prototype permite crear nuevos objetos a partir de la clonación de instancias existentes, evitando la creación directa mediante constructores complejos. Es especialmente útil cuando la creación de objetos implica procesos costosos o cuando se desea replicar estructuras existentes.

En el sistema desarrollado, este patrón se utiliza para gestionar la generación de copias de entregas. Se decidió incorporarlo debido a la necesidad de mantener versiones de una entrega sin alterar el objeto original, lo cual es fundamental para la trazabilidad y control de cambios.

La implementación se refleja en la clase “CopiaEntrega”, la cual permite generar una copia de una instancia de “Entrega”. En lugar de crear una nueva entrega desde cero, se replica el estado del objeto existente, incluyendo su contenido, archivo asociado y metadatos. Esto asegura consistencia entre versiones y simplifica la lógica de duplicación dentro del sistema.

4. Singleton: El patrón Singleton garantiza que una clase tenga una única instancia durante toda la ejecución del sistema.

En este sistema, se decidió aplicar este patrón a la clase “BackEndFachada”, ya que representa el núcleo del sistema y coordina todas las operaciones. Tener varias instancias de esta clase podría generar inconsistencias en el manejo del estado y en la coordinación de los servicios.

La implementación se evidencia mediante atributos estáticos como “getInstance()”, los cuales controlan la creación única de la fachada. Además, el acceso

a la instancia se realiza a través de un método estático, evitando la instanciación directa desde otras clases. De esta manera, se asegura que toda la interacción con el backend se realice a través de una única instancia controlada.

Capítulo 3

Conclusiones

La arquitectura propuesta permitió estructurar el sistema de forma clara y organizada mediante una separación en capas: presentación, lógica de negocio y persistencia. Esta división favorece el desacoplamiento entre componentes y facilita la mantenibilidad del sistema.

El backend cumple un rol fundamental dentro de la arquitectura al actuar como una fachada central que concentra la lógica de negocio. Este enfoque simplifica la interacción de los clientes, ya que estos no necesitan conocer los detalles internos del sistema, reduciendo así el acoplamiento.

Además, la integración de componentes externos, como el servicio de control de versiones (Git), junto con el sistema de base de datos, fortalece la arquitectura al incorporar mecanismos de trazabilidad y gestión de cambios. Esto permite que el sistema no solo gestione información, sino que también registre su evolución de manera estructurada.

La definición clara de interfaces entre componentes, como las rutas `/api`, `/data` y `/git`, permite una comunicación estructurada y bien delimitada entre las distintas capas del sistema. Esto permite tener claro los puntos de riesgo durante el funcionamiento y comunicación de los componentes.

Por otra parte, el diseño de la capa de persistencia, normalizado hasta Cuarta Forma Normal, garantiza la integridad, consistencia y eliminación de redundancias en los datos, asegurando un manejo confiable de la información.

Finalmente, la aplicación de patrones de diseño contribuye a una arquitectura dividida y evolutiva, permitiendo organizar la lógica del sistema de manera más eficiente. En conjunto, estas decisiones proporcionan una base sólida y escalable que permite la evolución futura del sistema.

Referencias

- Gruenhagen, J. H., Sinclair, P. M., Carroll, J.-A., Baker, P. R. A., Wilson, A., y Demant, D. (2024). The rapid rise of generative AI and its implications for academic integrity: Students' perceptions and use of chatbots for assistance with assessments. *Computers and Education: Artificial Intelligence*. (Early access)
- Laturette, K., Syahrani, A., y Octavica, L. (2025). Using artificial intelligence can increase academic fraud in generation Z. *Jurnal Akuntansi & Auditing Indonesia*, 29(1), 102–118. Descargado de <https://doi.org/10.20885/jaai.vol29.iss1.art9> (License: CC BY-SA 4.0) doi: 10.20885/jaai.vol29.iss1.art9
- Turnitin. (2026). *How turnitin works*. Descargado de <https://www.turnitin.com/> (Consultado el 27 de abril de 2026)
- United Nations Educational, Scientific and Cultural Organization. (2023). *AI and the future of education: Disruptions, dilemmas and directions* (Inf. Téc.). Paris, France: UNESCO. Descargado de <https://www.unesco.org/en/articles/ai-and-future-education-disruptions-dilemmas-and-directions> (Licensed under CC-BY-SA 3.0 IGO)